TELANGANA
CYBER SECURITY BUREAU

08-March-2026

PENETRATION TESTING (VAPT) REPORT

Security test report on "Vuln_Lab" Web application

Prepared By: **M. Samba Siva Rao**

1. Executive Summary

A security assessment was conducted on the “Vuln_Lab” web application to identify potential vulnerabilities. During this test, a multiple security issues and identified verse critical vulnerability including XSS, Authentication bypass, SQL Injection, IDOR, etc. Which may allow unauthorized access to sensitive data.

- SQL Injection
- IDOR
- Cross-Site Scripting

The findings indicate systemic security weaknesses due to insecure configurations, and lack of secure coding practices.

---

2. Methodology

The assessment followed a structured methodology based on the **OWASP Web Security Testing Guide (WSTG)** and industry-standard penetration testing practices.

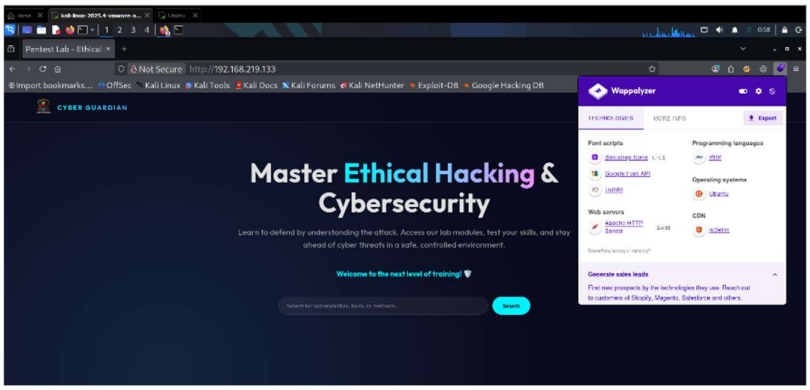
2.1 Reconnaissance

2.1.1 Technology Stack Disclosure via Wappalyzer

Description :

During the reconnaissance phase of the web application penetration test, the technology stack used by the target application was identified using the browser extension Wappalyzer.

Technology fingerprinting allows an attacker to determine the frameworks, libraries, server software, and programming languages used by a web application.



- Apache HTTP Server (version 2.4.58)

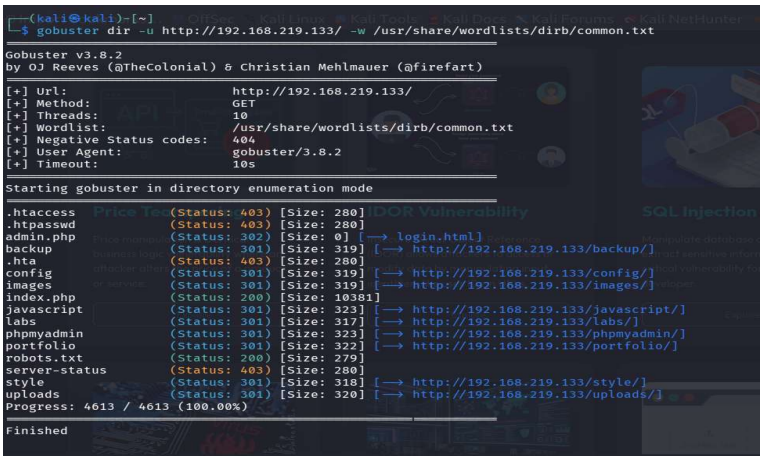
This information disclosure may assist attackers in performing targeted attacks against known vulnerabilities in these technologies.

2.1.2 Finding: Sensitive Directory and File Enumeration

Severity: Medium

During the reconnaissance phase, directory brute-forcing was performed against the target web application to discover hidden or unlinked directories and files.

The tool **Gobuster** was used to enumerate directories using the wordlist **common.txt**. The scan revealed several directories and files that are accessible or discoverable by an attacker.



```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.219.133/ -w /usr/share/wordlists/dirb/common.txt
═══════════════════════════════════════════════════════════════
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
═══════════════════════════════════════════════════════════════
[+] Url:                     http://192.168.219.133/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
═══════════════════════════════════════════════════════════════
Starting gobuster in directory enumeration mode
═══════════════════════════════════════════════════════════════
.htaccess            (Status: 403) [Size: 280]
.htpasswd            (Status: 403) [Size: 280]
admin.php            (Status: 302) [Size: 0] [--> login.html]
backup               (Status: 301) [Size: 319] [--> http://192.168.219.133/backup/]
.hta                 (Status: 403) [Size: 280]
config               (Status: 301) [Size: 319] [--> http://192.168.219.133/config/]
images               (Status: 301) [Size: 319] [--> http://192.168.219.133/images/]
index.php            (Status: 200) [Size: 10381]
javascript           (Status: 301) [Size: 323] [--> http://192.168.219.133/javascript/]
labs                 (Status: 301) [Size: 317] [--> http://192.168.219.133/labs/]
phpmyadmin           (Status: 301) [Size: 323] [--> http://192.168.219.133/phpmyadmin/]
portfolio            (Status: 301) [Size: 322] [--> http://192.168.219.133/portfolio/]
robots.txt           (Status: 200) [Size: 279]
server-status        (Status: 403) [Size: 280]
style                (Status: 301) [Size: 318] [--> http://192.168.219.133/style/]
uploads              (Status: 301) [Size: 320] [--> http://192.168.219.133/uploads/]
Progress: 4613 / 4613 (100.00%)
═══════════════════════════════════════════════════════════════
Finished
```

- /config/
- /backup/
- /images/
- /phpmyadmin/
- /uploads/

These directories were discovered through brute-force enumeration and **some of them were accessible via the browser without authentication.**

Impact

The exposure of internal directories can assist attackers in further exploitation. Potential risks include:

- Discovery of administrative interfaces (e.g., /admin.php)
- Access to configuration files
- Exposure of backup files
- Access to uploaded user content
- Identification of vulnerable applications such as **phpMyAdmin**

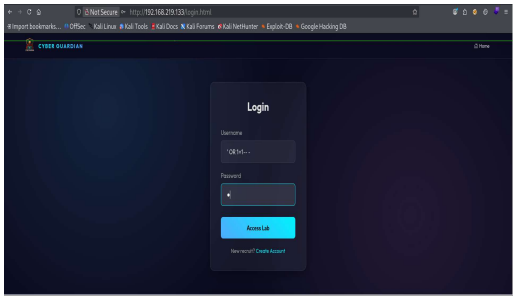
2.2 Authentication testing

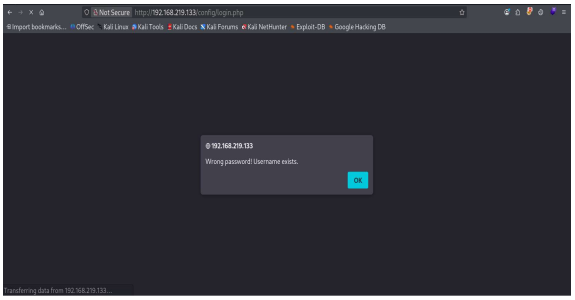
Finding: SQL Injection in Authentication Mechanism

Severity: High

Description

During authentication testing, the login functionality of the target web application was tested for input validation weaknesses. The tester attempted to inject a malicious SQL payload into the login form fields to determine whether the backend database query properly sanitizes user input.





Request → **Response**

Screenshots provided show the payload injected into the login form and the resulting server response.

Proof of Concept (PoC)

During manual testing, the following SQL payload was injected into the username field:

' OR 1=1-- -

Steps performed:

1. Navigate to the login page.
2. Enter the SQL payload in the Username field.
3. Enter any value in the Password field.
4. Submit the login form.

The application responded with the message:

Wrong password! Username exists.

This behavior indicates that the injected payload altered the backend SQL query logic and confirmed that the username condition was evaluated as true, demonstrating improper input handling.

Affected Endpoint

http://192.168.219.133/login.html

Impact

Successful exploitation of SQL Injection vulnerabilities can allow attackers to:

- Bypass authentication controls
- Extract sensitive data from the database
- Modify or delete database records
- Enumerate database structure
- Execute administrative operations on the database

Recommendation

To prevent SQL Injection vulnerabilities, the following measures should be implemented:

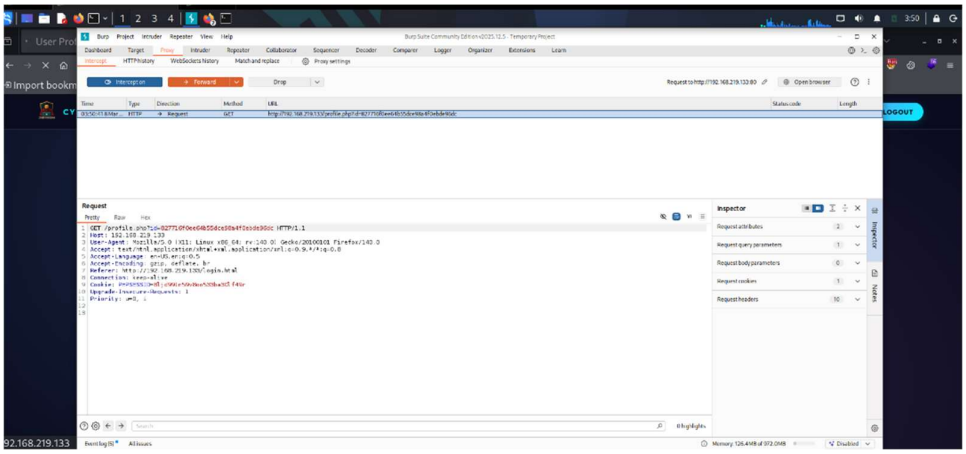
1. Use **parameterized queries (prepared statements)** for all database operations.
2. Implement strict **input validation and sanitization**.
3. Avoid directly concatenating user input into SQL queries.
4. Implement proper error handling to avoid revealing database behavior.

2.3 Finding: IDOR via Predictable User Identifier

Severity: High

Description

I was observed that the application identifies users using a value generated from the (user ID + Salt valu) combined with a hashing algorithm. The implementation appears to rely on the **MD5** algorithm with an additional salt value to generate identifiers used in application requests



The use of MD5 for generating user identifiers is not recommended due to its predictable nature and known weaknesses. A stronger hashing algorithm such as SHA-256 should be used in combination with proper authorization checks

---

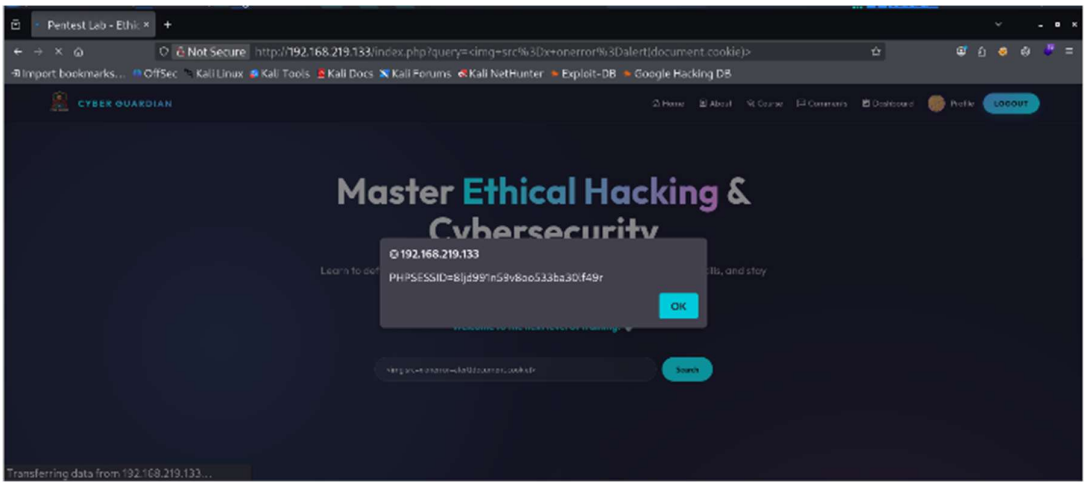
2.4 Finding: Cross-Site Scripting (XSS) via Search Function

Severity: High

Description

During testing of the search functionality, I was observed that the application attempts to filter malicious input by blocking the <script> tag. However, the filtering mechanism is incomplete and does not prevent other HTML-based JavaScript execution techniques.

By injecting a malicious payload into the search input field, it was possible to execute JavaScript code in the victim's browser using an image tag with an event handler



Screenshots show the injected payload and the resulting JavaScript execution.

Proof of Concept (PoC)

The application blocks <script> tags but fails to filter other HTML event handlers.

The following payload was injected into the search input field:

<img src=x onerror=alert(document.cookie)>

Steps performed:

1. Navigate to the search functionality.
2. Enter the malicious payload in the search input field.
3. Submit the request.
4. The injected payload is rendered in the response page.

The alert dialog displays the value of document.cookie, confirming that JavaScript execution is possible and session data can be accessed.

Impact

Successful exploitation of an XSS vulnerability may allow attackers to:

- Steal user session cookies
- Hijack authenticated user sessions
- Perform actions on behalf of the victim
- Redirect users to malicious websites
- Deliver phishing attacks
- Inject malicious scripts into the application

Recommendation

To mitigate Cross-Site Scripting vulnerabilities, the following security measures should be implemented:

1. Implement **proper input validation and output encoding**.
2. Do not rely on simple keyword filtering (e.g., blocking <script> only).
3. Encode all user-controlled data before rendering it in HTML.
4. Implement a **Content Security Policy (CSP)** to restrict JavaScript execution.

3. Vulnerability Summary Table

s.no	Vulnerability	Severity	Status	End Points
1	SQL Injection	Critical	Found	192.168.219.133/login.html
2	IDOR	Medium	Found	192.168.219.133/profile.php?id=
3	Cross-Site Scripting (XSS)	High	Found	192.168.219.133/index.php?query=
4	Insecure file upload	-	Not Found	-
5	SSRF	-	Not Found	-
6	Security Misconfiguration	Medium	Found	192.168.219.133/config/

Conclusion

In this assessment, I identified a number of security issues of varying severity levels. These vulnerabilities may allow unauthorized access, data exposure, or other potential risks if exploited by malicious actors. Additionally, several common web vulnerabilities were tested but were not detected in the application, indicating that certain security controls are properly implemented.

It is recommended that the identified vulnerabilities be addressed as a priority, especially those classified as high or medium severity. Implementing secure coding practices, improving input validation, strengthening access control mechanisms, and regularly performing security assessments will significantly improve the security posture of the application.